System Design: Scaling & Reliability Patterns

Detailed interview-oriented guide covering all 21 topics from your list

Mental model: Don't add a technology just because it is popular. First identify the bottleneck or requirement, then choose the pattern that directly addresses it. Every scaling technique introduces trade-offs.

Quick Decision Map

Technique	Primary purpose	Key trade-off / watch-out
High reads	Caching / Read Replicas / Indexing	Staleness, replication lag, index write cost
Write bottleneck	Batching / Async / Sharding	Latency, ordering, distributed complexity
Decoupling	Queues / Events / Pub/Sub	Duplicates, ordering, eventual consistency
Long-running work	Workers / Job Queues / Workflows	Retries, state, duplicate execution
Failures	Backoff / Timeouts / Circuit Breakers / Idempotency	Retry storms, partial failures
Data distribution	Partitioning / Sharding / Consistent Hashing	Hotspots, rebalancing, cross-partition queries
Real-time	WebSockets / SSE / Long Polling	Connections, scaling, network behavior

1. Caching

What is it? Stores frequently accessed data closer to the application to avoid repeated expensive database/service reads.

How it works / key ideas

- Typical flow: Client → API → Cache → Database. A cache hit returns immediately; a miss reads the source and populates the cache.
- Common policies include TTL, LRU/LFU eviction, cache-aside, write-through, write-back, and refresh-ahead.
- Useful for profiles, catalogs, configuration, permissions, computed results, and expensive API responses.
- Distributed caches such as Redis let multiple application instances share cached data.

Use when reads are frequent and repeated, and some staleness is acceptable.

Trade-offs / pitfalls: Watch for stale data, invalidation, cache stampedes, hot keys, memory limits, and cache outages.

Interview cue: Interview cue: Ask whether the data can tolerate staleness before introducing a cache.

2. Read Replicas

What is it? Copies data from a primary database to one or more replicas so read traffic can be distributed.

How it works / key ideas

- Writes normally go to the primary while read-only queries can be routed to replicas.
- This increases aggregate read throughput and reduces primary read load.
- Asynchronous replication can cause replication lag and stale reads.
- Read-after-write consistency may require routing a user's immediate reads to the primary.

Use for read-heavy workloads where the database's read capacity is the bottleneck.

Trade-offs / pitfalls: Replicas do not solve a write-capacity bottleneck; failover and replication lag also need consideration.

Interview cue: Interview cue: Separate read scaling from write scaling.

3. Indexing

What is it? Adds a data structure that helps the database locate matching rows without scanning the whole table.

How it works / key ideas

- Indexes can make selective queries much faster.
- Composite indexes cover multiple columns; column order matters.
- Indexes consume storage and add work to inserts, updates, and deletes.
- Indexes should follow actual query patterns and access paths.

Use when slow queries are caused by inefficient scans or lookups.

Trade-offs / pitfalls: Avoid blindly indexing every column; consider selectivity, query shape, and write overhead.

Interview cue: Interview cue: Before sharding, check whether query/index design is the real bottleneck.

Technique	Primary purpose	Key trade-off / watch-out
Indexing	Make individual queries faster	Storage + write overhead
Caching	Avoid repeated reads	Invalidation / staleness
Read replicas	Scale aggregate reads	Replication lag

4. Batching

What is it? Groups multiple operations into one request or processing unit.

How it works / key ideas

- Reduces network round trips and per-request overhead.
- Useful for bulk writes, ingestion, analytics, notifications, and APIs that support batch requests.
- Bound batch size by count and/or time to avoid excessive latency and memory usage.
- Large batches increase failure blast radius.

Use when many small operations create avoidable overhead.

Trade-offs / pitfalls: Trade off throughput against latency, partial failures, memory, and API batch-size limits.

Interview cue: Interview cue: If individual writes are expensive mostly because of per-request overhead, batching may be enough.

5. Async Processing

What is it? Moves non-critical work out of the synchronous request path.

How it works / key ideas

- Example: create an order synchronously, then process email, analytics, or recommendations asynchronously.
- A queue/event broker often connects the API and workers.
- The caller gets lower latency, while downstream work completes later.
- This commonly introduces eventual consistency.

Use for expensive work that does not need to finish before the user response.

Trade-offs / pitfalls: Handle retries, duplicates, ordering, observability, and user-facing status carefully.

Interview cue: Interview cue: Identify what must happen before the response and what can happen later.

6. Sharding

What is it? Splits a large logical dataset across multiple database instances.

How it works / key ideas

- A shard key determines where a record lives.
- Good shard keys distribute load and match important query patterns.
- Sharding can increase write and storage capacity.
- Cross-shard joins and transactions become harder; re-sharding is operationally significant.

Use when one database node cannot provide enough write throughput, storage, or overall capacity.

Trade-offs / pitfalls: Watch for hot shards, cross-shard queries, distributed transactions, rebalancing, and shard-key mistakes.

Interview cue: Interview cue: Explain why read replicas are insufficient when writes are the bottleneck.

Technique	Primary purpose	Key trade-off / watch-out
Read replicas	Scale reads	Writes still hit primary
Partitioning	Split data logically	Cross-partition access
Sharding	Distribute data across nodes	Distributed complexity

7. Message Queues

What is it? Buffers messages between producers and consumers so they can operate independently.

How it works / key ideas

- Producer → Queue → Consumer/Worker.
- Queues absorb spikes and smooth workload.
- Consumers can scale horizontally based on backlog.
- Important concepts: acknowledgements, visibility timeouts, retries, dead-letter queues, and delivery semantics.

Use for buffering, asynchronous work, workload smoothing, and service decoupling.

Trade-offs / pitfalls: At-least-once delivery can produce duplicates, so consumers should be idempotent where necessary.

Interview cue: Interview cue: A queue usually distributes work; it is different from broadcast-style pub/sub.

8. Event-Driven Architecture

What is it? Services communicate through events representing facts that happened, such as OrderCreated or PaymentCompleted.

How it works / key ideas

- A producer publishes an event without synchronously calling every downstream consumer.
- Independent consumers react to the same business event.
- This reduces direct coupling and makes new consumers easier to add.
- Event schemas, ownership, versioning, replay, ordering, and observability matter.

Use when multiple independent systems need to react to the same business events.

Trade-offs / pitfalls: Expect eventual consistency and design for duplicates, ordering, schema evolution, and debugging.

Interview cue: Interview cue: Treat an event as a durable business fact, not merely another HTTP call.

9. Pub/Sub

What is it? Publishes messages to a topic so multiple independent subscribers can receive the event.

How it works / key ideas

- Designed for fan-out: analytics, notifications, fraud, fulfillment, etc. can independently consume the same event.
- Subscribers process messages at different rates.
- Some systems support retention and replay.
- Each subscriber can have its own processing and failure behavior.

Use when one event needs to be independently consumed by multiple downstream systems.

Trade-offs / pitfalls: Watch consumer lag, retention, duplicates, ordering, and schema compatibility.

Interview cue: Interview cue: One worker doing one task suggests a queue; many independent consumers suggest pub/sub.

10. Background Workers

What is it? Dedicated processes that execute asynchronous tasks outside the main request path.

How it works / key ideas

- Typical flow: API → Job Queue → Worker Pool → Database/External Service.
- Workers scale horizontally based on queue depth or processing latency.
- Workers need graceful shutdown, concurrency limits, timeouts, retries, and dead-letter handling.
- They are useful for CPU-heavy, I/O-heavy, and long-running work.

Use when deferred work needs dedicated compute capacity.

Trade-offs / pitfalls: Watch for stuck jobs, duplicate execution, worker crashes, starvation, and resource contention.

Interview cue: Interview cue: The queue stores work; the worker executes it.

11. Job Queues

What is it? Represent units of deferred work that workers execute later.

How it works / key ideas

- Jobs can contain an operation plus the minimum data required to execute it.
- Useful features include priority, delay, scheduling, retries, deadlines, and dead-letter queues.
- Idempotent handlers protect against duplicate execution.
- Useful metrics include queue depth, processing rate, and oldest-job age.

Use for reliable deferred execution, retries, scheduling, and controlled worker concurrency.

Trade-offs / pitfalls: Watch duplicate jobs, poison jobs, priority starvation, and backlog growth.

Interview cue: Interview cue: Ask what happens if a worker crashes after doing the side effect but before acknowledgement.

12. Workflow Engines

What is it? Coordinate multi-step, stateful processes that can run for minutes, hours, or days.

How it works / key ideas

- A workflow can model steps, branching, parallel work, timers, retries, compensation, and human actions.
- The engine persists workflow state so execution can resume after failures.
- Unlike a simple job queue, it models the overall process and its state.
- Useful for approvals, sagas, scheduled processes, and long-running business workflows.

Use when the process has multiple durable steps, timers, state transitions, or recovery requirements.

Trade-offs / pitfalls: Watch workflow complexity, versioning, orchestration coupling, and implementation leakage.

Interview cue: Interview cue: If you need to remember exactly where a multi-step process stopped, consider a workflow engine.

13. Retries with Backoff

What is it? Reattempts failed operations while increasing the delay between attempts.

How it works / key ideas

- Exponential backoff can use delays such as 100 ms, 200 ms, 400 ms, 800 ms, with a cap.
- Jitter randomizes retry timing and reduces synchronized retry spikes.
- Retries are mainly appropriate for transient failures.
- Bound retries by attempt count or deadline and pair retries with idempotency when side effects can repeat.

Use when a failure is likely temporary and the operation is safe to retry.

Trade-offs / pitfalls: Avoid retry storms, cascading overload, retrying permanent failures, and duplicate side effects.

Interview cue: Interview cue: State what is retried, how often, how many times, and what happens after the final failure.

14. Timeouts

What is it? Places a bound on how long a caller waits for a dependency or operation.

How it works / key ideas

- Without timeouts, stuck calls can consume threads, connections, or worker slots.
- Connection, request, and end-to-end deadlines can be separate.
- A timeout does not guarantee the remote operation stopped.
- Timeouts therefore need to be considered with retries and idempotency.

Use when dependencies can hang or respond too slowly.

Trade-offs / pitfalls: Timeouts that are too short cause false failures; timeouts that are too long exhaust resources.

Interview cue: Interview cue: Use bounded deadlines and propagate them downstream where appropriate.

15. Circuit Breakers

What is it? Stops repeatedly calling a dependency that is failing or overloaded.

How it works / key ideas

- Closed: calls flow normally.
- Open: calls fail fast for a recovery period.
- Half-open: limited test calls determine whether recovery occurred.
- Circuit breakers reduce cascading failure and resource waste.

Use when a dependency can remain unhealthy long enough that continued calls would worsen the situation.

Trade-offs / pitfalls: Tune thresholds carefully and combine with timeouts, monitoring, fallbacks, and bounded retries.

Interview cue: Interview cue: Timeout limits how long a call waits; a circuit breaker controls whether calls should be attempted.

16. Idempotency

What is it? Ensures repeating the same logical operation does not create additional unintended side effects.

How it works / key ideas

- An idempotency key is commonly used for payments or order creation.
- The server stores the key and result; a repeated key can return the original result.
- Important when clients retry after timeouts or messages are delivered more than once.
- The key must be scoped and retained for an appropriate period.

Use for side-effecting operations that may be retried or duplicated.

Trade-offs / pitfalls: Watch key collisions, insufficient retention, inconsistent state, and non-atomic side effects.

Interview cue: Interview cue: Ask what happens if the server succeeds but the client times out before receiving the response.

17. Partitioning

What is it? Divides a dataset into smaller logical pieces.

How it works / key ideas

- Can use range, hash, list, time, geography, tenant, or another business dimension.
- Time partitioning can make recent queries scan less data.
- Partition pruning lets the database skip irrelevant partitions.
- Partitioning can improve maintenance and query performance without necessarily using separate servers.

Use when a dataset is very large and access/maintenance can be isolated by a natural partition key.

Trade-offs / pitfalls: Watch for skew, poor keys, cross-partition queries, and too many tiny partitions.

Interview cue: Interview cue: Partitioning is the broad idea of dividing data; sharding commonly distributes those partitions across nodes.

18. Consistent Hashing

What is it? Maps keys to nodes while minimizing key movement when nodes join or leave.

How it works / key ideas

- A hash ring maps keys to nodes.
- Adding/removing a node changes ownership for only part of the keyspace.
- Virtual nodes improve distribution.
- Useful in distributed caches and key-value/routing systems.
- It is a distribution technique, not a complete consistency or replication mechanism.

Use when a changing set of nodes needs to own a large keyspace with limited redistribution.

Trade-offs / pitfalls: Watch for uneven distribution, hotspots, node churn, and replication/failure strategy.

Interview cue: Interview cue: Commonly appears when designing a horizontally scaled distributed cache.

19. WebSockets

What is it? Provides a persistent, bidirectional connection between client and server.

How it works / key ideas

- After connection establishment, either side can send messages without a new HTTP request for every update.
- Useful for chat, collaboration, multiplayer interactions, and interactive live dashboards.
- Large connection counts require connection management and capacity planning.
- Heartbeats and reconnect logic detect dead connections and recover from network changes.

Use when both client and server need low-latency, continuous communication.

Trade-offs / pitfalls: Watch connection scaling, reconnect storms, authentication, load balancers, backpressure, and horizontal fan-out.

Interview cue: Interview cue: Choose when bidirectional persistent communication is actually required.

Technique	Primary purpose	Key trade-off / watch-out
WebSockets	Bidirectional real-time	Connection/state scaling
SSE	Server → client streaming	Unidirectional
Long polling	Near-real-time over HTTP	More request overhead

20. Server-Sent Events (SSE)

What is it? Keeps an HTTP connection open so the server can stream events to the client.

How it works / key ideas

- Communication is primarily server → client.
- Useful for notifications, live feeds, progress updates, and dashboards.
- The client can still use normal HTTP for commands.
- SSE can be simpler than WebSockets when bidirectional communication is unnecessary.

Use for server-to-client streaming over HTTP.

Trade-offs / pitfalls: Watch proxy buffering, long-lived connection limits, authentication, reconnects, and network interruptions.

Interview cue: Interview cue: If only the server needs to push updates, SSE is a natural option to consider.

21. Long Polling

What is it? The client sends an HTTP request that the server holds until an update occurs or a timeout is reached.

How it works / key ideas

- When an event occurs, the server returns the response and the client immediately opens another request.
- Provides near-real-time behavior without requiring a persistent bidirectional protocol.
- Works with ordinary HTTP infrastructure but has more request overhead than persistent streaming.
- At scale, connection and timeout management matter.

Use when persistent streaming protocols are difficult and near-real-time delivery is sufficient.

Trade-offs / pitfalls: Watch repeated HTTP overhead, timeouts, load balancer limits, and reconnect behavior.

Interview cue: Interview cue: Compare it with SSE/WebSockets based on latency, infrastructure compatibility, and communication direction.

How to choose the right pattern in a system-design interview

Start from the symptom, not the technology.

1. Identify the bottleneck

- Reads slow / database overloaded → investigate query patterns, indexing, caching, and read replicas.
- Writes slow → investigate batching and async processing; consider sharding when one node is genuinely at its capacity limit.
- Traffic spikes → use queues to buffer and workers to drain the backlog.
- Services tightly coupled → consider asynchronous messaging or event-driven communication.
- Long-running process → background workers; multi-step durable state → workflow engine.
- Repeated dependency failures → timeouts, bounded retries with backoff/jitter, circuit breakers, and idempotency.
- Huge distributed keyspace → partition/shard data and choose an appropriate distribution strategy.
- Live updates → choose WebSockets, SSE, or long polling based on communication direction and infrastructure.

2. Ask the consistency question

Can the system tolerate stale or eventually consistent data? This often determines whether caching, replicas, asynchronous processing, or events are appropriate.

3. Ask the failure question

For every network call or asynchronous job: What if it times out? What if it runs twice? What if the dependency is down for 10 minutes? What if a worker crashes after the side effect but before acknowledgement?

4. Ask the scale question

Estimate request rate, data size, read/write ratio, payload size, latency targets, and growth. Then choose the simplest mechanism that solves the calculated bottleneck.

Final cheat sheet

Technique	Primary purpose	Key trade-off / watch-out
Repeated reads	Caching	Reduce repeated backing-store reads
Read-heavy DB	Read replicas	Scale read capacity
Slow queries	Indexing	Avoid unnecessary scans
Many tiny writes	Batching	Reduce per-operation overhead
Slow non-critical work	Async processing	Move work off request path
DB capacity ceiling	Sharding	Distribute data/write load
Service decoupling	Queue / events	Reduce synchronous coupling
Fan-out	Pub/Sub	Independent consumers
Deferred execution	Workers / job queues	Process later
Multi-step process	Workflow engine	Persist process state

Technique	Primary purpose	Key trade-off / watch-out
Transient failure	Retry + backoff	Recover safely
Slow dependency	Timeout	Bound waiting
Failing dependency	Circuit breaker	Fail fast
Duplicate delivery	Idempotency	Prevent duplicate side effects
Large dataset	Partitioning	Divide data logically
Changing nodes	Consistent hashing	Minimize data movement
Bidirectional real-time	WebSockets	Two-way persistent channel
Server push	SSE	HTTP streaming
Near-real-time HTTP	Long polling	Simple compatibility